



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



PRACTICA 8

NOMBRE COMPLETO: Mora Ortega Judith

Nº de Cuenta: 318006538

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: Mayo/2026

CALIFICACIÓN: _____

Desarrollo de práctica

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Agregar luz de tipo spotlight para la nave de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.

Vamos a empezar cambiando todas funciones para adaptarlas a Nave.

```
Model Nave_M;
```

```
Nave_M = Model(); Nave_M.LoadModel("Models/nave.obj");
```

```
// — Nave —  
model = glm::mat4(1.0f);  
model = glm::translate(model, navePosition);  
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));  
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);  
Nave_M.RenderModel();
```

Cambiamos el estado de la nave, para que cambie de luz con un bool dependiendo si es la parte positiva.

```
// — Estado de la nave —  
glm::vec3 navePosition = glm::vec3(0.0f, 5.0f, 6.0f);  
const float NAVE_SPEED = 3.0f;  
bool naveAvanzando = false; // se mueve en X negativa (tecla J)  
bool naveRetrocediendo = false; // se mueve en X positiva (tecla L)
```

Las teclas que vamos a usar son : **I (Adelante)**, **J(Atrás)** y la **J (X Negativa)**, **L (X Positiva)**

```
//— Input de la nave —————
void ProcessNaveInput(bool* keys, GLfloat dt)
{
    if (keys[GLFW_KEY_I]) navePosition.z -= NAVE_SPEED * dt;
    if (keys[GLFW_KEY_K]) navePosition.z += NAVE_SPEED * dt;

    naveAvanzando = false;
    naveRetrocediendo = false;

    if (keys[GLFW_KEY_J]) {
        navePosition.x -= NAVE_SPEED * dt;
        naveAvanzando = true;
    }
    if (keys[GLFW_KEY_L]) {
        navePosition.x += NAVE_SPEED * dt;
        naveRetrocediendo = true;
    }
}
```

También cambie para prender y apagar la lámpara por la tecla “M”

```
// — Input de la lámpara —————
void ProcessLampInput(bool* keys, GLfloat dt)
{
    static bool lPressed = false;
    if (keys[GLFW_KEY_M] && !lPressed) {
        lampOn = !lampOn;
        lPressed = true;
    }
    if (!keys[GLFW_KEY_M]) lPressed = false;
}
```

Los colores para cada una de las luces.

```
// Spotlight nave - ADELANTE (amarillo)
spotLights[3] = SpotLight(1.0f, 1.0f, 0.0f,
    0.0f, 0.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
// Spotlight nave - ATRÁS (azul)
spotLights[4] = SpotLight(0.0f, 0.5f, 1.0f,
    0.0f, 0.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
```

El funcionamiento de SpotLights

```
// Spotlights de la nave
glm::vec3 naveLightPos = navePosition + glm::vec3(0.0f, -0.5f, 0.0f);

glm::vec3 naveDirFwd = glm::normalize(glm::vec3(-0.5f, -1.0f, 0.0f));
spotLights[3].SetFlash(naveLightPos, naveDirFwd);
spotLights[3] = SpotLight(
    1.0f, 1.0f, 0.0f,
    0.0f, naveAvanzando ? 2.0f : 0.0f,
    naveLightPos.x, naveLightPos.y, naveLightPos.z,
    naveDirFwd.x, naveDirFwd.y, naveDirFwd.z,
    1.0f, 0.0f, 0.0f,
    20.0f);

glm::vec3 naveDirBwd = glm::normalize(glm::vec3(0.5f, -1.0f, 0.0f));
spotLights[4].SetFlash(naveLightPos, naveDirBwd);
spotLights[4] = SpotLight(
    0.0f, 0.5f, 1.0f,
    0.0f, naveRetrocediendo ? 2.0f : 0.0f,
    naveLightPos.x, naveLightPos.y, naveLightPos.z,
    naveDirBwd.x, naveDirBwd.y, naveDirBwd.z,
    1.0f, 0.0f, 0.0f,
    20.0f);
```

2.- Crear o instanciar una pecera (traslucida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.

Para este paso, vamos a declarar e instanciar rectángulo, y una nueva textura.

Declaración de Pez Abisal

```
// — Modelos Pez Abisal —
Model PezCuerpo_M;
Model PezAntena_M;
Model PezFoco_M;
```

Declaración de pecera

```
CrearPecera(); // meshList [6,7]
```

```

// — Pecera: caja con normales hacia adentro + tapa superior con agua —
void CrearPecera()
{
    // 5 caras: fondo, frente, atrás, izquierda, derecha
    // Normales apuntan hacia el INTERIOR (+Y, +Z, -Z, +X, -X)
    unsigned int pec_indices[] = {
        0,1,2, 2,3,0,    // fondo
        4,5,6, 6,7,4,    // frente
        8,9,10, 10,11,8, // atrás
        12,13,14, 14,15,12, // izquierda
        16,17,18, 18,19,16 // derecha
    };
};

```

```

GLfloat pec_vertices[] = {
    // fondo (y=-1) — normal hacia arriba (+Y)
    -1.0f,-1.0f,-1.0f, 0.0f,0.0f, 0.0f,1.0f,0.0f,
    1.0f,-1.0f,-1.0f, 1.0f,0.0f, 0.0f,1.0f,0.0f,
    1.0f,-1.0f, 1.0f, 1.0f,1.0f, 0.0f,1.0f,0.0f,
    -1.0f,-1.0f, 1.0f, 0.0f,1.0f, 0.0f,1.0f,0.0f,
    // frente (z=+1) — normal hacia adentro (-Z)
    -1.0f,-1.0f, 1.0f, 0.0f,0.0f, 0.0f,0.0f,-1.0f,
    1.0f,-1.0f, 1.0f, 1.0f,0.0f, 0.0f,0.0f,-1.0f,
    1.0f, 1.0f, 1.0f, 1.0f,1.0f, 0.0f,0.0f,-1.0f,
    -1.0f, 1.0f, 1.0f, 0.0f,1.0f, 0.0f,0.0f,-1.0f,
    // atrás (z=-1) — normal hacia adentro (+Z)
    -1.0f,-1.0f,-1.0f, 0.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f,-1.0f,-1.0f, 1.0f,0.0f, 0.0f,0.0f,1.0f,
    1.0f, 1.0f,-1.0f, 1.0f,1.0f, 0.0f,0.0f,1.0f,
    -1.0f, 1.0f,-1.0f, 0.0f,1.0f, 0.0f,0.0f,1.0f,
    // izquierda (x=-1) — normal hacia adentro (+X)
    -1.0f,-1.0f,-1.0f, 0.0f,0.0f, 1.0f,0.0f,0.0f,
    -1.0f,-1.0f, 1.0f, 1.0f,0.0f, 1.0f,0.0f,0.0f,
    -1.0f, 1.0f, 1.0f, 1.0f,1.0f, 1.0f,0.0f,0.0f,
    -1.0f, 1.0f,-1.0f, 0.0f,1.0f, 1.0f,0.0f,0.0f,
    // derecha (x=+1) — normal hacia adentro (-X)
    1.0f,-1.0f,-1.0f, 0.0f,0.0f, -1.0f,0.0f,0.0f,
    1.0f,-1.0f, 1.0f, 1.0f,0.0f, -1.0f,0.0f,0.0f,
    1.0f, 1.0f, 1.0f, 1.0f,1.0f, -1.0f,0.0f,0.0f,
    1.0f, 1.0f,-1.0f, 0.0f,1.0f, -1.0f,0.0f,0.0f,
};
};

```

```

// Tapa superior con textura de agua - normal hacia adentro (-Y)
unsigned int agua_indices[] = { 0,1,2, 2,3,0 };
GLfloat agua_vertices[] = {
    -1.0f, 1.0f, -1.0f,  0.0f, 0.0f,  0.0f, -1.0f, 0.0f,
     1.0f, 1.0f, -1.0f,  1.0f, 0.0f,  0.0f, -1.0f, 0.0f,
     1.0f, 1.0f,  1.0f,  1.0f, 1.0f,  0.0f, -1.0f, 0.0f,
    -1.0f, 1.0f,  1.0f,  0.0f, 1.0f,  0.0f, -1.0f, 0.0f,
};

Mesh* pecera = new Mesh();
pecera->CreateMesh(pec_vertices, pec_indices, 160, 30);
meshList.push_back(pecera); // [6] - paredes translúcidas

Mesh* agua = new Mesh();
agua->CreateMesh(agua_vertices, agua_indices, 32, 6);
meshList.push_back(agua); // [7] - tapa superior con agua
}

```

Texturas de usar

```

aguaTexture = Texture("Textures/agua.jpg");          aguaTexture.LoadTextureA();
vidrioTexture = Texture("Textures/vidrio.jpg");        vidrioTexture.LoadTextureA();

```

Pez

```

// — PEZ ABISAL —————
glm::mat4 pezBase = glm::mat4(1.0f);
pezBase = glm::translate(pezBase, pezPosition);
pezBase = glm::scale(pezBase, glm::vec3(0.03f, 0.03f, 0.03f));

// Cuerpo principal
color = glm::vec3(0.2f, 0.2f, 0.35f); // gris azulado abismal
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(pezBase));
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
PezCuerpo_M.RenderModel();

// Antena (hereda la misma matriz base que el cuerpo)
color = glm::vec3(0.8f, 0.7f, 0.1f); // amarillo bioluminiscente
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
modelaux = pezBase;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
PezAntena_M.RenderModel();

// Foco en la punta de la antena
color = glm::vec3(1.0f, 1.0f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
modelaux = pezBase;
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 9.8f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
PezFoco_M.RenderModel();

```

Percera

```
// — PECERA: paredes de vidrio + tapa de agua —
glm::vec3 pec_center = glm::vec3(-10.0f, 0.0f, 5.0f);
glm::vec3 pec_scale = glm::vec3(2.0f, 1.0f, 2.0f); // ancho, alto, profundo

glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE);

// — Paredes (fondo, frente, atrás, izquierda, derecha) - textura VIDRIO —
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::mat4(1.0f);
model = glm::translate(model, pec_center);
model = glm::scale(model, pec_scale);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
vidrioTexture.UseTexture();
meshList[6]->RenderMesh();

// — Tapa superior - textura AGUA translúcida —
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::mat4(1.0f);
model = glm::translate(model, pec_center);
model = glm::scale(model, pec_scale);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
aguaTexture.UseTexture();
meshList[7]->RenderMesh();
```

```
glDepthMask(GL_TRUE);
glDisable(GL_BLEND);
```

3.- Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.

Se cambió como lo pedía la práctica.

```
// Lámpara (pointLight[1]) – tecla L
if (lampOn) {
    pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
                                0.0f, 1.0f,
                                0.0f, -4.0f, -10.0f,
                                0.5f, 0.3f, 0.1f);
}
else {
    pointLights[1] = PointLight(0.0f, 0.0f, 0.0f,
                                0.0f, 0.0f,
                                0.0f, -4.0f, -10.0f,
                                0.5f, 0.3f, 0.1f);
}
```

Las teclas que vamos a usar para el pez; U, N,B

```
// — Input Pez Abisal —————
void ProcessPezInput(bool* keys, GLfloat dt)
{
    if (keys[GLFW_KEY_U]) {
        pezPosition.x -= PEZ_SPEED * dt;
        pezPosition.y += PEZ_SPEED * dt;
    }
    if (keys[GLFW_KEY_N]) {
        pezPosition.x += PEZ_SPEED * dt;
        pezPosition.y -= PEZ_SPEED * dt;
    }
}
```

```
// Tecla B – prende/apaga luz puntual del bulbo
static bool bPressed = false;
if (keys[GLFW_KEY_B] && !bPressed) {
    pezLuzOn = !pezLuzOn;
    bPressed = true;
}
if (!keys[GLFW_KEY_B]) bPressed = false;
```

4. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y y eje z.

Las teclas que vamos a usar para el pez; 1,2,3,3,4,5,6


```

// Rotación del spotlight en X (teclas 1 y 2)
if (keys[GLFW_KEY_Z]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(0.0f, SPOT_ROT_SPEED * dt, 0.0f));
if (keys[GLFW_KEY_X]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(0.0f, -SPOT_ROT_SPEED * dt, 0.0f));

// Rotación del spotlight en Y (teclas 3 y 4)
if (keys[GLFW_KEY_V]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(SPOT_ROT_SPEED * dt, 0.0f, 0.0f));
if (keys[GLFW_KEY_G]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(-SPOT_ROT_SPEED * dt, 0.0f, 0.0f));

// Rotación del spotlight en Z (teclas 5 y 6)
if (keys[GLFW_KEY_Y]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(0.0f, 0.0f, SPOT_ROT_SPEED * dt));
if (keys[GLFW_KEY_T]) pezSpotDir = glm::normalize(pezSpotDir + glm::vec3(0.0f, 0.0f, -SPOT_ROT_SPEED * dt));

```

La luces que pide este punto vamos a declarar

```

// Posición del bulbo - misma para ambas luces
glm::vec3 bulboPos = pezPosition + glm::vec3(0.0f, 9.8f * 0.03f, 5.0f * 0.03f);

// Luz puntual AZUL del bulbo - ilumina en todas direcciones
if (pezLuzOn) {
    pointLights[2] = PointLight(0.0f, 0.3f, 1.0f,    // azul
                                0.0f, 1.5f,
                                bulboPos.x, bulboPos.y, bulboPos.z,
                                0.5f, 0.3f, 0.1f);
}
else {
    pointLights[2] = PointLight(0.0f, 0.0f, 0.0f,    // apagada
                                0.0f, 0.0f,
                                bulboPos.x, bulboPos.y, bulboPos.z,
                                0.5f, 0.3f, 0.1f);
}

```

```

// Spotlight NARANJA del bulbo -
spotLights[5].SetFlash(bulboPos, pezSpotDir);
if (pezLuzOn) {
    spotLights[5] = SpotLight(1.0f, 0.4f, 0.0f,    // naranja
                              0.0f, 2.0f,
                              bulboPos.x, bulboPos.y, bulboPos.z,
                              pezSpotDir.x, pezSpotDir.y, pezSpotDir.z,
                              1.0f, 0.0f, 0.0f, 25.0f);
}
else {
    spotLights[5] = SpotLight(0.0f, 0.0f, 0.0f,    // apagada
                              0.0f, 0.0f,
                              bulboPos.x, bulboPos.y, bulboPos.z,
                              pezSpotDir.x, pezSpotDir.y, pezSpotDir.z,
                              1.0f, 0.0f, 0.0f, 25.0f);
}

```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

1. Tuve problemas con la parte de pecera porque es difícil, normalización para pueda reflejar la luz
2. Y me costó que el pez tuviera las dos luces, cambien lo lampara.
3. La tarde cambiar helicóptero → nave

3.- Conclusión

Durante el desarrollo de esta práctica se presentaron diversas dificultades. La implementación de la pecera resultó compleja debido al manejo de transparencia y la normalización correcta de las paredes para que reflejaran la luz adecuadamente. Lograr que el pez abisal tuviera sus dos luces funcionales (puntual azul y spotlight naranja) también representó un reto, al igual que la configuración correcta de la lámpara. Finalmente, la transición del helicóptero a la nave requirió ajustes en los modelos y su iluminación asociada. A pesar de estas dificultades, se logró una escena funcional con transparencia, iluminación dinámica y movimiento controlado.

4.-Bibliografía en formato APA

Roque, J. (2026). *opengl3.3ydosmain* [Material de código]. UMAN.